

104. Maximum Depth of Binary Tree

Problem Summary

Given the root of a binary tree, return its maximum depth — the number of nodes along the longest path from root to the farthest leaf.

Approach

Post-order recursion. Get the depth of left and right subtrees, return the larger one plus 1 for the current node.

Algo

1. If root == null return 0.
2. lt = maxDepth(root.left).
3. rt = maxDepth(root.right).
4. Return Math.max(lt, rt) + 1.

Observations

- Identical to height() helper used in LC 543 and LC 110 — same function, different problem name.
- BFS alternative: count number of levels using level-order traversal.
- Recursion depth = tree height — skewed tree causes O(n) stack depth.

Time Complexity & Space Complexity

- O(n) — every node visited once.
- O(h) — recursion stack. O(log n) balanced, O(n) skewed.

Key Insights

- Depth and height are the same thing here — both measure longest root-to-leaf path.
- +1 counts the current node itself.
- Pure post-order — left first, right second, combine at root.
- Base case null → 0 means leaf nodes return max(0,0)+1 = 1 automatically.

Problem List

104. Maximum Depth of Binary Tree

Solved

Easy

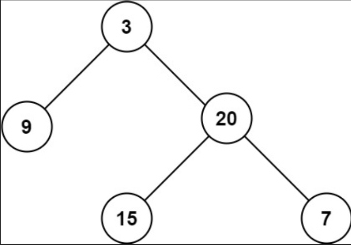
Topics

Companies

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input:

root = [3,9,20,null,null,15,7]

Output:

3

Example 2:

Input:

root = [1,null,2]

Output:

2

Constraints:

The number of nodes in the tree is in the range [0, 10⁴].

Accepted

40 / 40 testcases passed

Sahil Khan submitted at Jul 30, 2026 17:44

Runtime

0 ms | Beats 100.00%

Memory

47.06 MB | Beats 22.78%

Code | Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
10 */
11 class Solution {
12     public int maxDepth(TreeNode root) {
13         if(root==null) return 0;
14
15         int lt = maxDepth(root.left);
16         int rt = maxDepth(root.right);
17
18         return Math.max(lt, rt)+1;
19     }
20 }
```

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Input

root = [3,9,20,null,null,15,7]

Output